

Zero Trust Architecture: From Conceptual Foundations to System-Level Enforcement

RIVALDO DEMLABING TANDAH, Westphalian University of Applied Sciences, Germany

Zero Trust (ZT) replaces the perimeter-based security model with a simple but demanding principle: never trust, always verify. Realizing it across heterogeneous infrastructures raises open questions at every layer. This report traces Zero Trust Architecture (ZTA) through three complementary contributions and one practical demonstration. The survey by Syed et al. consolidates the conceptual foundations of ZTA, its logical components as standardized in NIST SP 800-207, and the state of the art in continuous authentication, dynamic access control, and micro-segmentation. Ma et al. address IoT authentication with FAST-SM9, a lightweight identity-based scheme with dynamic re-authentication that cuts latency by 56.6% and energy by 63% against PKI baselines. Hong et al. extend ZTA into the operating system with SysFlow, enforcing flow-based policies inside the Linux kernel at under 5% overhead. Finally, a self-built four-VM laboratory operationalizes four ZT mechanisms multi-factor authentication, risk-based re-authentication, micro-segmentation, and kernel-level enforcement and contains a simulated breach end to end, documented with live screenshots and released as a reproducible artifact (Appendix B). Together, they show Zero Trust shifting from a way to design networks into a property that whole systems can enforce and measure.

Additional Key Words and Phrases: Zero Trust Architecture, NIST SP 800-207, Continuous Authentication, SM9, Identity-Based Cryptography, Micro-Segmentation, eBPF, Linux Security Modules, Policy Enforcement Point.

Preface

This report was prepared as part of the course “Emerging Challenges in Cybersecurity Research”, supervised by Prof. Dr.-Ing. Tobias Urban, in the summer semester 2026, at Westphalian University of Applied Sciences, in the Department of Computer Science and Communication.

The technical demonstration in Section 6 is documented with the screenshots captured live during the run, a dedicated step by step reproduction guide (Appendix A), and a cleaned, fully working source code appendix (Appendix B) so that a third party can rebuild the four VM laboratory and re-run the attack chain without access to the author’s machines.

Author’s Contact Information: Rivaldo Demlabing Tandah, rivaldo.demlabing-tandah@studmail.w-hs.de, Westphalian University of Applied Sciences, Gelsenkirchen, Germany.

1 Introduction and Motivation

In the past, corporate security was based on a “castle-and-moat” model: the idea was to build a solid wall around the network, inspect everything passing through it, and trust what was inside. This model no longer works with modern infrastructure: the cloud, mobile working, containers, microservices and billions of IoT devices. Data flows between servers, employees connect from anywhere, and a single mistake can allow a hacker to gain access to the system.

Zero Trust (ZT) starts from the opposite assumption, and not a comfortable one: treat the breach as something that has already happened. Named by Kindervag in 2010 [5], popularized by Google’s BeyondCorp [13], and standardized by NIST SP 800-207 [9], Zero Trust Architecture (ZTA) removes implicit trust from network location and makes every access decision explicit, contextual, and continuously re-evaluated. The principle *never trust, always verify* is easy to state, but realizing it raises hard questions: how can identity be verified continuously without overwhelming users or devices? How can decisions incorporate risk and context in real time? And how far *down* the stack from the network to the kernel must enforcement reach before the principle is complete?

This report examines that landscape along three guiding questions. **RQ1**: what does a complete Zero Trust Architecture consist of, and which building blocks are mature? **RQ2**: can continuous, Zero-Trust-grade *authentication* be made affordable for the most constrained devices? **RQ3**: how can Zero Trust *enforcement* be extended beyond the network, into the operating system? Three peer-reviewed publications and one practical demonstration answer these in turn and form a deliberate progression from concept to enforcement. The comprehensive survey by Syed et al. [11] (Section 3) consolidates the conceptual core of ZTA and shows that most building blocks exist but neither scale down to constrained devices nor reach into end systems. Ma et al. [6] (Section 4) close the first gap at the identity layer of the IoT with FAST-SM9, a lightweight identity-based scheme with dynamic re-authentication for devices with as little as 264 KB of RAM. Hong et al. [4] (Section 5) close the second at the enforcement layer with SysFlow, which carries Zero Trust into the Linux kernel. Finally, a self-built laboratory demonstration (Section 6) operationalizes four ZT mechanisms multi-factor authentication, computed risk-based re-authentication, micro-segmentation, and kernel-level file protection and traces how they contain a simulated multi-stage attack, its limitations illustrating why the research problems are genuinely hard.

The remainder is organized as follows. Section 2 gives the background (the NIST ZTA model, identity-based cryptography, and the Linux kernel mechanisms). Sections 3–5 summarize and appraise the three papers, Section 6 presents the technical demonstration, and Section 7 concludes with a cross-cutting synthesis and outlook.

2 Background

This section introduces the concepts used throughout the report: the NIST Zero Trust reference architecture, the identity-based cryptography underlying FAST-SM9, and the Linux kernel facilities on which SysFlow and the demonstration build.

2.1 From Perimeter Security to Zero Trust

Traditional network security splits the world into an untrusted outside and a trusted inside, separated by firewalls, DMZs, and VPN gateways. It fails in two ways: it is blind to *lateral movement* once an attacker gains any internal foothold, internal systems accept its connections without further scrutiny and “inside” has eroded, as cloud workloads, home offices, unmanaged devices, and short-lived containers all blur the perimeter. Zero Trust inverts the default: no request is trusted for *where* it originates; every request must be authenticated, authorized, and encrypted, with trust re-evaluated continuously [5, 9].

2.2 The NIST SP 800-207 Reference Model

NIST SP 800-207 [9] defines the vocabulary in which all three papers and the demonstration can be expressed. Access control is split into a logical *control plane* and a *data plane*, as illustrated in Figure 1. Three components cooperate:

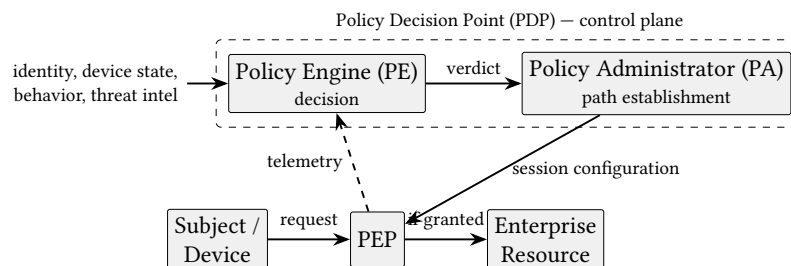


Fig. 1. Core logical components of a Zero Trust Architecture according to NIST SP 800-207 [9]. Every request crosses a Policy Enforcement Point; decisions are computed dynamically in the control plane.

The **Policy Engine (PE)** makes the access decision, evaluating enterprise policy together with dynamic signals identity, device state, behavioral analytics, threat intelligence into a grant, deny, or revoke verdict, often summarized as a *trust* or *risk score*. The **Policy Administrator (PA)** executes that decision by establishing or tearing down the communication path (e.g., issuing session-specific credentials), while the **Policy Enforcement Point (PEP)** sits on the data path and enforces the verdict on every request. PE and PA together form the *Policy Decision Point (PDP)*.

Two consequences recur throughout the report. Enforcement is only as fine-grained as the PEP, a PEP that sees only network flows cannot control what happens *inside* a host (the observation

motivating SysFlow) and, because every request traverses PDP logic, the performance of decision and enforcement becomes a first-class problem (the central concern of FAST-SM9 for constrained devices). SP 800-207 also formulates the tenets the literature operationalizes: every resource protected, all communication secured regardless of location, access per-session and least-privilege, decisions dynamic, and posture continuously monitored. *Micro-segmentation* applies these tenets to the topology, wrapping small groups of resources in their own enforcement boundary so a compromised node keeps only the connectivity it demonstrably needs.

2.3 Identity-Based Cryptography and SM9

Classical Public Key Infrastructures (PKI) bind identities to keys through certificates, whose issuance, validation, and revocation are costly certificate chains add hundreds of bytes and multiple verifications per authentication, which for MCU-class IoT dominates the energy budget. *Identity-based cryptography* (IBC), conceived by Shamir and made practical by Boneh and Franklin [3], removes certificates: an identity string (a device ID or fingerprint hash) is the public key, and a trusted Key Generation Center (KGC) derives the matching private key from a master secret.

SM9 [10], the Chinese national IBC standard, realizes this over elliptic-curve groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ of prime order N linked by a bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, whose evaluation dominates the cost. Two properties matter for Section 4: identities *are* the public keys, so no certificate transport is needed (attractive for IoT), yet the expensive pairings make a naive deployment too slow for constrained hardware the starting point for FAST-SM9.

2.4 System-Level Enforcement: LSM and eBPF

The Linux kernel offers two facilities that make operating-system-level Zero Trust enforcement practical. The *Linux Security Modules* (LSM) framework [14] places hooks on security-relevant kernel operations opening files, checking inode permissions, mapping memory, connecting sockets and allows a security module to allow or deny each operation. SELinux and AppArmor are the best-known LSM consumers; SysFlow uses the same hooks as the interception layer of its data plane. *eBPF* (the in-kernel extended Berkeley Packet Filter) complements LSM with programmability: verified bytecode can be loaded into an in-kernel virtual machine at runtime and JIT-compiled to near-native speed, without loading kernel modules or rebooting. Together, the two mechanisms provide exactly what a system-level PEP requires: complete mediation of resource accesses (LSM) and safe, dynamic extensibility of the enforcement logic (eBPF).

Three smaller primitives complete the toolkit used in the demonstration: *TOTP* (RFC 6238 [7]) derives short-lived, replay-resistant codes from a shared secret and the current time; *risk-based (step-up) authentication* turns contextual signals (new geolocation, VPN, unknown device) into a numeric score and re-challenges only above a threshold, realizing the “dynamic policy” tenet

with minimal friction; and *cyber deception* (honeypots) redirects an access to worthless data while alerting defenders a richer response than allow/deny that both SysFlow and the demonstration use.

3 Paper 1: Zero Trust Architecture (ZTA)—A Comprehensive Survey

The first paper, published by Syed et al. in *IEEE Access* [11], is a systematic survey of the Zero Trust research landscape. Its goal is twofold: to give practitioners and researchers a consolidated map of the mechanisms that a ZTA is composed of, and to identify where the mechanisms available in the literature fall short of the Zero Trust ideal. The survey is organized along the functional pillars of a ZTA authentication, access control, encryption, segmentation, and automation and closes with research directions that, as Sections 4 and 5 will show, later work has begun to address.

3.1 Scope and Architecture Model

The authors first ground the survey in the NIST SP 800-207 component model introduced in Section 2.2 and depicted in Figure 1: a Policy Engine and Policy Administrator in the control plane, Policy Enforcement Points on the data path, fed by supporting systems such as continuous diagnostics, threat intelligence, identity management, and SIEM. They emphasize that ZTA is not a product but an architectural philosophy that can be realized through different implementation techniques identity-governance-centric, micro-segmentation-centric, or based on software-defined perimeters (SDP) and that practical deployments combine all three. The survey's implicit thesis is that the maturity of a ZTA deployment is bounded by the weakest of its pillars, which motivates the pillar-by-pillar review.

3.2 Authentication

The longest part of the survey treats authentication, the entry point of every Zero Trust decision, along four strands. *Conventional user authentication* (passwords, tokens, biometrics) is structurally insufficient alone: static credentials establish trust once, at session start, contradicting continuous verification; multi-factor authentication raises the bar but stays episodic. *Context-aware authentication* augments credentials with signals such as geolocation, device identity, and access history, letting the PE distinguish a routine login from an anomalous one and demand step-up selectively at the cost of a tension between signal richness and privacy. *Continuous authentication* re-establishes identity throughout the session via behavioral biometrics (keystroke and mouse dynamics) or ambient signals, with accuracy, energy cost, and user acceptance as open problems exactly the space FAST-SM9's dynamic re-authentication addresses for machine-to-machine settings. *Device authentication* matters because in IoT most principals are not people; techniques range from certificate-based identity to physically unclonable functions and fingerprinting, and

the survey notes that PKI-style certificate management scales poorly to constrained devices—the gap identity-based schemes such as SM9 target.

3.3 Access Control

The second pillar reviews access-control models against Zero Trust requirements (Table 1). Identity- and role-based control (RBAC) are simple and auditable but static encoding *who* may access *what*, not *when*. Attribute-based control (ABAC) generalizes to predicates over subject, object, and environment attributes, and risk-based control (RbAC) makes the decision a function of a computed risk score, so the same request is granted at noon from a managed laptop and denied at midnight from an unknown network. The survey also covers usage control (UCON) the classical model closest to per-request evaluation and blockchain-based policy management, whose latency the authors view skeptically for interactive decisions. The trajectory from static roles to continuous, risk-weighted evaluation mirrors the shift from one-time to continuous authentication.

Table 1. Access-control models reviewed by the survey [11] and their fit to Zero Trust tenets.

Model	Decision basis	Dynamics	Zero Trust fit
IBAC	individual identity	static	entry-level; no context
RBAC	role membership	static	auditable, coarse-grained
ABAC	subject/object/env. attributes	per request	good; policy complexity
RbAC	computed risk score	adaptive	high; needs calibrated scores
UCON	mutable attributes, obligations	continuous	closest to per-session tenet
Blockchain-based	decentralized ledger policy	transaction-bound	open; latency concerns

3.4 Encryption and Constrained Devices

Because Zero Trust mandates that all communication be protected regardless of location, a pillar is devoted to *lightweight* cryptography for IoT: small-state block ciphers, short-key elliptic-curve schemes, and mutual-authentication protocols that minimize round trips. The authors stress that the cryptographic *cost* of continuous verification not just protocol design decides feasibility on battery-powered devices, making the link to Paper 2 explicit: FAST-SM9 pushes that cost below the threshold at which continuous re-authentication becomes affordable.

3.5 Micro-Segmentation and Software Defined Perimeters

On the network layer, the survey reviews micro-segmentation host-, hypervisor-, and network-fabric-based as the mechanism that shrinks trust zones and blocks lateral movement, and software-defined perimeters, which render resources invisible until a client has authenticated (“authenticate first, connect second”), typically via single-packet authorization. Both techniques implement the

least-privilege tenet at the connectivity level. The authors observe that policy management is the practical bottleneck: fine-grained segments multiply rules, and static rule sets erode as workloads move an operational argument for the programmable, dynamically updatable enforcement that Paper 3 later provides at the system level.

3.6 Automation, Orchestration, and Trust Computation

The final pillar surveys the control loop that keeps a ZTA adaptive: threat-intelligence feeds, device-state monitoring, security-situation awareness, and machine learning for anomaly detection and policy automation. Particular attention is paid to *trust computation*: algorithms that aggregate heterogeneous signals into the score on which the Policy Engine acts. The survey classifies scoring approaches and notes their shared weaknesses calibration, explainability, and vulnerability to adversarial manipulation of input signals.

3.7 Identified Challenges and Appraisal

The survey closes with research directions: continuous authentication schemes that are accurate yet cheap; access-control models that scale to dynamic, containerized workloads; micro-segmentation with automated, verifiable policy management; and migration of ZTA cryptography toward post-quantum primitives [8].

Where the paper earns its place is in the consolidation itself: it takes a discourse thick with vendor language and reorganizes it into a research map anchored in the NIST model, and it stays honest about the parts that are still immature. The weaknesses are the ones you would expect from a survey. It describes mechanisms without putting them side by side under a shared metric, and it says almost nothing about the system level, meaning the processes, files, and containers running on a host, which is exactly the blind spot Paper 3 sets out to fill. Being a 2022 snapshot, it is also thin on post-quantum migration and on eBPF-based enforcement, which is a fair measure of how quickly the field has moved since.

4 Paper 2: A Lightweight Zero-Trust Authentication Architecture for IoT via Unified Enhanced FAST-SM9 and Dynamic Re-Authentication

The second paper, by Ma et al. in *PLOS ONE* [6], takes up the challenge that the survey leaves open at the identity layer: making continuous, Zero-Trust-grade authentication affordable for the weakest devices in the infrastructure. The target environment is the cloud-edge-end IoT stack, where end devices are microcontrollers such as the Raspberry Pi Pico W with 264 KB of RAM and a battery budget, while the architecture must simultaneously scale up to millions of sessions at the cloud.

4.1 Problem Statement

The authors quantify why the two natural candidates fail here. PKI with RSA carries ~ 704 byte certificate chains per authentication, costs ~ 11.7 mJ per handshake, needs five seconds or more to recover from failures on MCU-class hardware, and offers no path to quantum resistance. Classical SM9 (Section 2.3) removes certificates, but its pairings push latency to 35–53 ms and 560 bytes per session and like PKI authenticate only *once*, at establishment. Neither provides the continuous re-verification Zero Trust demands, so the paper targets a scheme both lightweight enough for MCUs and continuous enough for Zero Trust.

4.2 FAST-SM9 and the Improved Single-Package Protocol

The first contribution is FAST-SM9, a unified enhancement of the SM9 signature scheme. Three optimizations carry most of the gain: pairing results and $\mathbb{G}_2$ elements that do not depend on the message are precomputed and cached, key-derivation steps are fused into a single pass through the Key Generation Center, and the scheme is exposed as one primitive usable identically by edge and end devices. The *Initial Authentication Phase* (IAP) then runs in six steps: the device sends its identity material (a hash of device fingerprint and hardware information, plus a timestamp); the edge derives and binds an OTP seed to this identity; the device derives its key, signs, and the edge verifies. The scheme remains standard SM9 at the mathematical core key extraction, signature (h, S) with $h = H_2(M \parallel w, N)$, and verification via pairing comparison—so existing security arguments for SM9 carry over.

On top of the primitive, the *Improved Single-Package protocol* (ISP) collapses the message flow: identity information, OTP seed, and signature travel in one consolidated transmission unit, and authentication and signature are fused into a single cryptographic operation. Eliminating the intermediate verification round-trips reduces the number of communication rounds by 40% without weakening integrity an important saving on lossy, low-bandwidth IoT links.

4.3 Zero-Trust Dynamic Re-Authentication

The second contribution turns the one-shot handshake into a continuously verified session, through four cooperating mechanisms:

- **DITGM (Dynamic Identity Token Generation).** After the IAP, each request carries a token $DIT = \text{HMAC}(\text{OTP_Seed} \parallel \text{Identity} \parallel \text{Timestamp})$. Tokens are time-bound, single-use, bound to the device fingerprint, and revocable through a cloud-propagated lock list the machine-to-machine analogue of the continuous authentication the survey called for.
- **Continuous Authentication Phase.** A monitor evaluates traffic and behavior telemetry at a 250 ms cadence, updates the trust score, and triggers either a lightweight ISP refresh or

a hard re-authentication. In the reported experiments, 94.1% of checks pass silently, 5.6% trigger a soft re-authentication, and 0.3% escalate to hard re-authentication or isolation.

- **FMM (Failure Management Mechanism).** A sliding-window failure counter per identity and IP locks out brute-force and credential-stuffing attempts, blocks the offending address, and raises SIEM alerts; lockouts expire automatically.
- **CMP (Communication Maintenance Phase).** A session key derived independently on both endpoints from the IAP transcript drives AES-GCM encryption of application data; bearer tokens authorize follow-up requests without re-running the IAP; and re-authentication is forced on token expiry, context change, prolonged inactivity, or access to sensitive resources.

Together these phases implement the NIST tenets per-session least privilege, dynamic decisions, continuous monitoring at the protocol level, for devices that cannot run a full policy stack. Figure 2 summarizes the resulting session lifecycle.

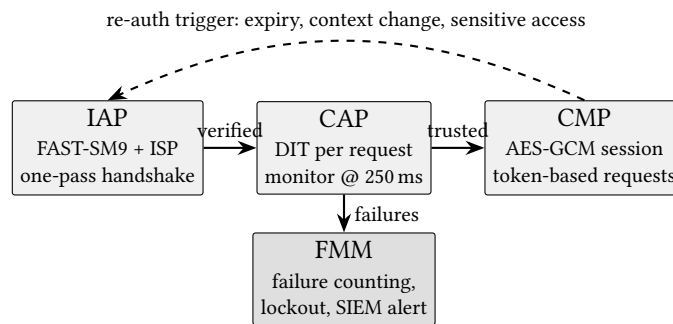


Fig. 2. Session lifecycle of the FAST-SM9 zero-trust architecture [6]: initial authentication (IAP), continuous authentication (CAP), communication maintenance (CMP), and failure management (FMM).

4.4 Evaluation

The evaluation spans two hardware classes: a Raspberry Pi 4B as edge-class device and a Pico W as constrained end device, with a workstation-hosted cloud/edge simulation and attack tooling (Kali Linux, Wireshark, Hping3). Table 2 compares the headline results, averaged over 100 runs, against the baselines evaluated in the paper: total authentication latency of 22.7 ms on the Pi 4B, a 56.6% reduction against the PKI-RSA baseline and 35.5% against classical SM9; communication cost of 464 bytes per authentication, of which only 16 bytes are the dynamic session token; energy of 4.2 mJ per authentication versus 11.7 mJ for PKI; and failure recovery within two seconds while being the only scheme in the comparison that supports continuous re-authentication at all.

Table 2. Performance comparison on the Raspberry Pi 4B, as reported by Ma et al. [6].

Metric	PKI-RSA	TinyECC	SM9 (base)	FAST-SM9
Authentication delay (ms)	52.3	27.4	35.2	22.7
Communication cost (bytes)	704	512	560	464
Energy per authentication (mJ)	11.7	5.8	6.1	4.2
Failure recovery (s)	≥ 5	≤ 3	≤ 4	≤ 2
Continuous re-authentication	no	no	no	yes
Formal verification (AVISPA)	—	—	—	yes

Penetration tests report 100% rejection of man-in-the-middle and replay attempts, sub-0.01% success for forged keys under partial key leakage, DoS isolation within ten seconds, and a side-channel (CPA) success rate of 1.7% with dynamic tokens versus 34.5% for static schemes. Formal validation complements the measurements: AVISPA [1] reports *SAFE* verdicts under both back-ends, and a Bellare Rogaway style argument [2] reduces session-key indistinguishability to the Bilinear Diffie–Hellman assumption, yielding EUF-CMA security and forward secrecy.

4.5 Appraisal

The paper’s strength is the completeness of its engineering argument: a concrete cryptographic optimization, a protocol exploiting it, a Zero Trust session layer, and an evaluation combining measurements, attacks, and formal analysis. The authors are candid about limitations. Gains shrink on the most constrained target the Pico W improves latency by only 11-12% because the Cortex-M0+ lacks pairing hardware, so the advantage is smallest where it is most needed. All experiments run in a localhost simulation (WAN latency, loss, and congestion unmodeled), and the PKI baseline is taken from prior literature rather than co-measured. “Quantum-aware” means stronger than RSA with a hybrid lattice option, not evaluated against NIST post-quantum schemes [8]. Finally, a competing scheme reaches 448 bytes, so FAST-SM9’s 464 are best read as the price of continuous re-authentication 16 bytes per request for Zero Trust semantics, a trade the authors argue is worthwhile.

5 Paper 3: SysFlow - Toward a Programmable Zero Trust Framework for System Security

The third paper, by Hong et al. in *IEEE Transactions on Information Forensics and Security* [4], attacks the blind spot noted in Section 3: existing ZTA verifies *network* access, but once a request has legitimately reached a host, the activities inside that host processes reading files, containers touching volumes, binaries mapping memory again enjoy implicit trust. A compromised microservice can therefore read a neighboring container’s secrets without ever crossing a network PEP.

SysFlow extends Zero Trust to this system level, and does so *programmably*, so that policies can follow the dynamics of container workloads.

5.1 Research Challenges and Architecture

The authors structure the problem as four challenges: (C1) abstract heterogeneous system activities into a uniform model; (C2) offer high-level programmability for dynamic workloads; (C3) provide context-aware visibility across hosts; and (C4) keep host overhead minor. The architecture mirrors software-defined networking (Figure 3). On every host the SysFlow *Data Plane* (SDP) is the PEP: an abstraction layer intercepts security-relevant operations and a flow-table manager matches them against rules. A logically centralized *Controller* (SC) is the PDP: it hosts security apps written against high-level APIs, compiles their policies into flow rules, and installs them over TLS. Events flow up, rules flow down. The threat model trusts the kernel, control channels, and controller; users, applications, and containers are untrusted.

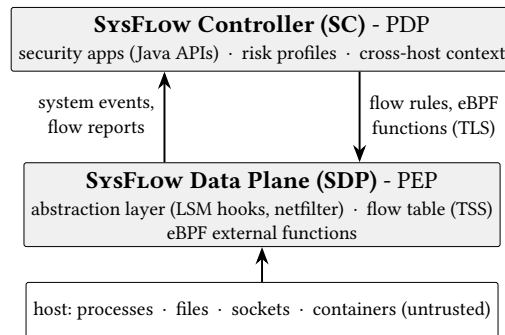


Fig. 3. SysFlow architecture [4]: an SDN-style split between a programmable controller (PDP) and a kernel-level data plane (PEP) on every host.

5.2 System Flow Abstraction

The conceptual core is the *system flow*: every system activity is modeled as a flow from a source entity to a destination resource. A flow rule is a triple $\langle match, action, priority \rangle$, where the match is itself a triple $\langle src, dst, type \rangle$ with wildcard support on every field, and actions compose in parallel ($a | a'$) or in sequence ($a \gg a'$). Beyond MAC-compatible allow and deny, the action set is deliberately rich: report and log for visibility; encode/decode to stamp and check contextual tags on network packets, enabling cross-host information-flow tracking; redirect to transparently divert a file access to a decoy by swapping the inode before file-descriptor binding; quarantine to isolate a process from a resource; and external to execute custom eBPF code in the data plane. Developers write semantic names (paths, process names); a binding service resolves them to stable

kernel identifiers (inodes) and keeps the mapping current as inodes and containers change. Rule lookup uses tuple-space search, giving constant-time matching and updates independent of the number of installed rules.

5.3 Programmability and Visibility

Policies use two models. In the *reactive* model the controller installs monitoring rules, receives reports, and responds with new rules no pre-configuration, ideal for tracking container instantiation, at the cost of a ~ 6.4 ms controller round trip. In the *proactive* model the logic is offloaded as a vetted eBPF external function into the data plane, cutting response latency to ~ 2.1 ms (a 60% reduction). Risk-aware response uses four profile templates of increasing generality: privilege-based (static allow/deny), context-based (over process/container/history attributes), scoring-threshold-based (e.g., EigenTrust-style scores), and fully custom logic. For visibility (C3), SDP context collectors gather process, file, and user context including a process file mapper- and share cross-host context reactively or via packet tags.

5.4 Implementation and Use Cases

The prototype comprises about 7,100 lines of C for the data plane and 4,300 of Java for the controller. Interception uses LSM hooks (file, inode, memory-mapping, socket) plus a dedicated hook before file-descriptor binding for redirection, and netfilter for the packet tagging LSM cannot do; namespace checks attribute events to containers. Six example applications show the expressiveness: risk-aware micro-segmentation (blocking, in the AtSea scenario, a compromised app server from reading the DB password, at $<5\%$ file-latency overhead); context-aware access control binding application users to system enforcement via eBPF, defeating an unpatched WordPress exploit (CVE-2019-8943) at $+4.2\%$; a file reflector redirecting `/etc/passwd` reads to a honey file with negligible latency; cross-layer data-leakage prevention via flow tagging; virtual patching of an nginx path-traversal bug without a restart; and time-based cross-host database access control.

5.5 Evaluation

Micro-benchmarks (LMBench) against unmodified Linux show the cost concentrated where mediation happens: file operations and memory mapping incur roughly 1–15% overhead (e.g., $+4.1\%$ file read, $+12.5\%$ open/close, $+12.8\%$ mmap), while the generic syscall path costs only $+0.56\%$ and socket throughput changes by less than 2%. Macro-benchmarks on real applications, summarized in Table 3, stay below 5% throughout.

Scalability experiments install 74,519 rules (every file on a fresh Ubuntu system) in 37.2 s, after which average file overhead is 4.7%; rule-modification latency stays constant at ~ 1.7 ms from 10,000 to 50,000 rules, confirming the constant-time flow table, and memory grows linearly at ~ 400 KB

Table 3. Macro-benchmark results for the SysFlow data plane with 1,000 installed rules, as reported by Hong et al. [4].

Workload	Metric	Baseline	SysFlow	Overhead
Web server (nginx, 500 concurrent)	requests/s	3,533	3,478	−1.6%
File transfer (wget, 1 GB)	completion time (s)	21.8	22.3	+2.3%
Database (MySQL, 1M records)	transactions/s	540.6	535.1	−1.0%
Microservice e-shop (4,093 users)	requests/s	2,472	2,401	−2.9%

per 1,000 rules. On the controller, a stress test with 223,496 rules keeps per-message processing below 0.4 ms, with 1 GB of RAM supporting on the order of 5,000 hosts at typical enterprise event rates.

5.6 Appraisal

SysFlow’s contribution is conceptual as much as technical: the SDN recipe a uniform flow abstraction, a programmable controller, a fast dumb data plane transfers to operating-system security, and Zero Trust semantics reach individual file accesses at single-digit-percent cost. The honest counterpoints: the controller is a central point of failure addressed only with standard hardening advice; the guarantees are empirical, with no formal verification of the reference monitor (named as future work); the implementation is Linux-only; and user/device identity the other half of Zero Trust is out of scope, so SysFlow must be paired with an authentication substrate such as Paper 2’s. This complementarity, more than any single result, is what makes the three papers cohere.

6 Technical Demonstration: Containing the Breach

The demonstration, presented on 23 June 2026, translates the preceding concepts into a working laboratory and answers one practical question: if an attacker gains a foothold behind the perimeter, do Zero Trust mechanisms built from ordinary open-source means actually contain the breach? Four controls are enforced for real, not mocked: multi-factor authentication, risk-based re-authentication, micro-segmentation, and kernel-level file protection with deception. The complete source and a step-by-step reproduction guide are supplied as a runnable artifact (Appendix B and A).

6.1 Laboratory Setup

The environment comprises four Ubuntu 22.04 virtual machines on an isolated NAT network (192.168.113.0/24): a user workstation (.131); a web server (.136) running a Flask CRM, “Atlas CRM”, that also hosts the PDP/PEP logic; a MySQL database behind UFW (.133); and a monitoring console (.134). There is deliberately no external identity provider, so every enforced control is visible in the

lab's own code. Figure 4 shows the enforced paths: micro-segmentation leaves the compromised web server exactly one necessary route.

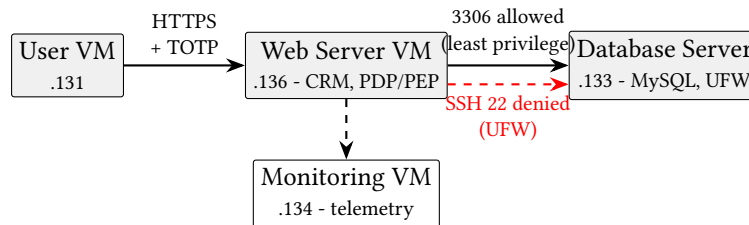


Fig. 4. Enforced communication paths in the demonstration network (isolated NAT, 192.168.113.0/24). Micro-segmentation leaves the compromised web server exactly one necessary pathway.

6.2 Attack Chain and Enforced Countermeasures

The demonstration follows a single narrative a legitimate session, a client-side compromise, and an attacker attempting to escalate with a Zero Trust control at each step (Table 4).

Stage 1 - verified entry. The user signs in with a hashed password and a TOTP code (RFC 6238 [7]), verified server-side by pyotp; wrong or expired codes are rejected, with no static bypass. This secures the front door but, in Zero Trust terms, establishes only *initial* trust.

Stage 2 - trust re-evaluated. When the session context changes, a risk engine recomputes a weighted score (baseline 12; +40 new country, +25 VPN, +20 new device, +5 new IP). In the run, clicking “Simulate context change” flipped the dashboard’s risk badge from 12/100 to 82/100 in front of the audience; the score crossed the step-up threshold of 70 and forced re-authentication rather than continuing silently, and only fell back to 15/100 once a fresh TOTP code was accepted at /reauth. This is risk-based access control (Section 3) in miniature, since the verdict is computed, not scripted.

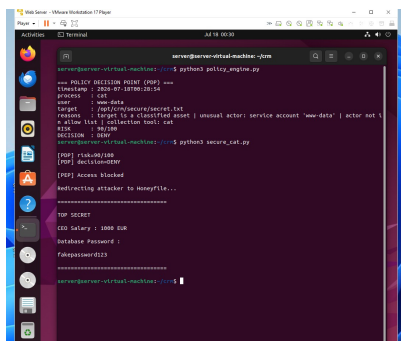
Stage 3 - blocked lateral movement. A phishing click yields a low-privilege www-data shell, but UFW denies SSH from the web server to the database while MySQL’s webuser holds only SELECT/INSERT/UPDATE/DELETE on the crm schema no CREATE USER, DROP, or mysql.*. Tellingly, the attacker’s ssh dbadmin@192.168.113.133 did not fail fast: it simply hung and eventually timed out, since a UFW deny silently drops the packets rather than refusing them while the one permitted mysql path returned rows instantly. The compromised host keeps exactly one necessary path the definition of micro-segmentation.

Stage 4 - kernel enforcement and deception. The attacker tries to read secret.txt. The rule-based PDP (policy_engine.py, in the spirit of SysFlow) scores the triple (cat, www-data, secret.txt) at 90/100 and denies it; the real file is root:600, so the kernel refuses the read regardless of any script; and the PEP wrapper secure_cat.py logs the attempt and serves a decoy

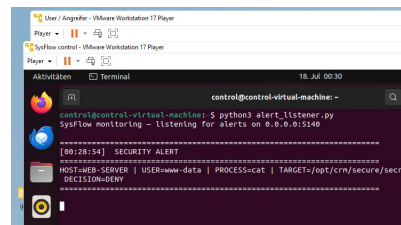
honeyfile the same deception primitive as SysFlow’s redirect action. Instead of the protected data, the shell printed the decoy’s fake secrets (“Database Password: fakepassword123”), so the attacker walks away believing the exfiltration succeeded. Every DENY verdict is streamed to a monitoring console (alert_listener.py) as a timestamped “SECURITY ALERT”: during the run it surfaced on the monitor over port 5140 within the same second the read was refused, closing the continuous-monitoring loop SP 800-207 requires and turning the four independent controls into one auditable narrative (Figure 5a).

Table 4. Attack chain and enforced Zero Trust countermeasures.

#	Attack vector	ZT mechanism	Tooling	Result
C1	Login attempt	MFA (real TOTP)	Flask, pyotp	Entry secured
C2	Context shift	Computed risk + step-up	Risk engine	Session paused
C3	Server compromise	Micro-segmentation	UFW, MySQL grants	Pivot failed
C4	Secret file access	Kernel perms + policy + decoy	root:600, PDP/PEP	Denied, deceived



(a) PDP/PEP on the web server: policy_engine.py returns RISK=90/DENY; secure_cat.py blocks the read and serves the honeyfile’s fake secrets.



(b) The monitoring VM’s alert_listener.py receiving the same DENY verdict in real time over port 5140.

Fig. 5. Stage 4 as captured live: (a) the denied secret read is redirected to a honeyfile, and (b) the verdict is simultaneously reported to the SOC console. The remaining stages are documented with screenshots in the reproduction guide (Appendix A).

6.3 Scope and Limitations

The demonstration proves containment of one chosen attack chain, not a general security property. The path is scripted and the controls were built knowing the trajectory, so there is no adversary emulation against an unprotected baseline (e.g., MITRE ATT&CK [12]); privilege escalation is

untested, since the attacker remains `www-data` and local `root` would disable UFW; and the policy engine is invoked manually rather than wired to live kernel telemetry, which would require syscall-level monitoring (auditd or eBPF, as in SysFlow). These gaps define the hardening roadmap. Even so, the essential result holds: four independent, genuinely enforced controls turned a perimeter breach into a contained incident that a third party can verify by re-execution (Appendix B, A).

7 Conclusion and Outlook

Taken together, the three papers and the demonstration cover the Zero Trust stack from concept to kernel, and their contributions are strikingly complementary; Table 5 maps each work onto the central NIST tenets. A few observations cut across all four contributions. Start with cost, which the survey singles out as the main obstacle to continuous verification. It no longer looks prohibitive: FAST-SM9 gets re-authentication down to 16-byte tokens on IoT hardware, and SysFlow enforces per-file-access policy at overheads in the single digits. What is left to solve is mostly a question of how to wire these pieces together, not how to afford them. *The PDP/PEP* split is the second thing worth noting. It turns up, essentially unchanged, at the protocol level, at the system level, and even inside a small Flask application. For a reference model that is a good sign: SP 800-207 has held up in practice. Both SysFlow and the demonstration point to a third idea. A defender working under an assume-breach premise needs more than a binary allow-or-deny. SysFlow’s redirect and quarantine, together with the honeyfile in the lab, let the defender absorb an intrusion and mislead the attacker rather than only block a request.

Table 5. Coverage of core Zero Trust tenets across the studied works.

Tenet (SP 800-207)	Survey [11]	FAST-SM9 [6]	SysFlow [4]	Demo
Verify explicitly	taxonomy	SM9 identity, ISP	app/user context	MFA (TOTP)
Continuous evaluation	concepts	DIT tokens, 250 ms	react./proact. rules	risk step-up
Least privilege	ABAC/RbAC models	per-session keys	flow rules, quarantine	UFW, SQL grants
Assume breach	motivation	FMM lockouts	deception, DLP	contained chain
Dynamic policy	trust computation	context triggers	programmable PDP	computed score

Against this backdrop the three research questions of Section 1 can be answered. **RQ1**: the survey by Syed et al. [11] established the conceptual frame and showed that in 2022 several of its pillars continuous authentication, dynamic access control, micro-segmentation were aspirational rather than deployed. **RQ2**: Ma et al. [6] answer affirmatively for the identity layer, making continuous, identity-based authentication affordable for microcontroller-class IoT devices (56.6% lower latency and 63% less energy than PKI). **RQ3**: Hong et al. [4] carry Zero Trust past the network boundary into the operating system, enforcing per-access policy in the Linux kernel at under 5% overhead. The demonstration assembled these into a working chain that contained a simulated breach, packaged as reproducible source (Appendix B) so its central claim can be re-executed.

The works share weaknesses that mark the field's frontier. Each depends on a central authority FAST-SM9's KGC and SysFlow's controller are single points of failure and their evaluations remain limited; only FAST-SM9 is machine-checked, and none tests post-quantum primitives [8]. The clearest next step is *integration*: one policy plane where device identity, session risk, and system-level enforcement inform one another, so risk continuously drives kernel-level rules. Pursued that far, Zero Trust becomes a measurable, enforceable property of entire systems.

References

- [1] Alessandro Armando, David Basin, Yohan Boichut, Yannick Chevalier, Luca Compagna, Jorge Cuéllar, Paul Hanks Drielsma, Pierre-Cyrille Héam, Olga Kouchnarenko, Jacopo Mantovani, Sebastian Mödersheim, David von Oheimb, Michaël Rusinowitch, Judson Santiago, Mathieu Turuani, Luca Viganò, and Laurent Vigneron. 2005. The AVISPA Tool for the Automated Validation of Internet Security Protocols and Applications. In *Proceedings of the 17th International Conference on Computer Aided Verification (CAV)*. 281–285. doi:10.1007/11513988_27
- [2] Mihir Bellare and Phillip Rogaway. 1993. Entity Authentication and Key Distribution. In *Advances in Cryptology — CRYPTO '93*. 232–249. doi:10.1007/3-540-48329-2_21
- [3] Dan Boneh and Matthew Franklin. 2001. Identity-Based Encryption from the Weil Pairing. In *Advances in Cryptology — CRYPTO 2001*. 213–229. doi:10.1007/3-540-44647-8_13
- [4] Sungmin Hong, Lei Xu, Jianwei Huang, Hongda Li, Hongxin Hu, and Guofei Gu. 2023. SysFlow: Toward a Programmable Zero Trust Framework for System Security. *IEEE Transactions on Information Forensics and Security* 18 (2023), 2794–2809. doi:10.1109/TIFS.2023.3264152
- [5] John Kindervag. 2010. *Build Security Into Your Network's DNA: The Zero Trust Network Architecture*. Technical Report. Forrester Research.
- [6] Zhaobin Ma, Hua Wei, Jun Jiang, Bin Wang, Hao Wang, and Zhenxin Di. 2025. A lightweight zero-trust authentication architecture for IoT via unified enhanced FAST-SM9 and dynamic re-authentication. *PLOS ONE* 20, 10 (2025), e0332943. doi:10.1371/journal.pone.0332943
- [7] David M'Raihi, Salah Machani, Mingliang Pei, and Johan Rydell. 2011. *TOTP: Time-Based One-Time Password Algorithm*. Technical Report RFC 6238. Internet Engineering Task Force. doi:10.17487/RFC6238
- [8] National Institute of Standards and Technology. 2024. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Technical Report FIPS 203. NIST. doi:10.6028/NIST.FIPS.203
- [9] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. 2020. *Zero Trust Architecture*. Technical Report NIST Special Publication 800-207. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-207
- [10] State Cryptography Administration of China. 2016. GM/T 0044-2016: SM9 Identity-Based Cryptographic Algorithms. Chinese National Cryptography Standard.
- [11] Naeem Firdous Syed, Syed W. Shah, Arash Shaghaghi, Adnan Anwar, Zubair Baig, and Robin Doss. 2022. Zero Trust Architecture (ZTA): A Comprehensive Survey. *IEEE Access* 10 (2022), 57143–57179. doi:10.1109/ACCESS.2022.3174679
- [12] The MITRE Corporation. 2026. MITRE ATT&CK: Enterprise Matrix. <https://attack.mitre.org>. Accessed: 2026-07-06.
- [13] Rory Ward and Betsy Beyer. 2014. BeyondCorp: A New Approach to Enterprise Security. *login*: 39, 6 (2014), 6–11.
- [14] Chris Wright, Crispin Cowan, Stephen Smalley, James Morris, and Greg Kroah-Hartman. 2002. Linux Security Modules: General Security Support for the Linux Kernel. In *Proceedings of the 11th USENIX Security Symposium*. 17–31.

A Reproducing the Demonstration

This appendix specifies, independently of the author’s machines, how to rebuild the four VM laboratory and reproduce the live run. The complete, unmodified source files are listed in Appendix B and supplied as a companion package.

The prerequisites are modest: any hypervisor able to host four Ubuntu 22.04 VMs on one isolated NAT/host-only network (VMware, VirtualBox, or a cloud VPC), Python 3.10+ with pip on each, and an authenticator app (or the bundled `otp_now.py`) enrolled with the shared TOTP secret. Assign the static addresses .131 (user), .136 (web server), .133 (database), .134 (monitor) and configure the three service VMs as follows; the UFW rules encode the micro-segmentation of Figure 4.

```
# --- Database VM (.133): least-privilege data path only ---
sudo apt update && sudo apt install -y mysql-server ufw
sudo mysql < db_setup.sql          # crm schema + 'webuser' (Appendix B.5)
sudo ufw allow from 192.168.113.136 to any port 3306 proto tcp
sudo ufw deny  from 192.168.113.136 to any port 22  proto tcp
sudo ufw allow 22/tcp              # SSH stays open from every OTHER source
sudo ufw enable

# --- Web Server VM (.136): CRM + PDP/PEP ---
sudo apt install -y python3-venv mysql-client
mkdir -p ~/crm/templates && cd ~/crm && python3 -m venv venv
./venv/bin/pip install flask pyotp werkzeug mysql-connector-python
# app.py + *.py -> ~/crm/ ; *.html -> ~/crm/templates/ (see Appendix B)
sudo ~/crm/venv/bin/python app.py 5000

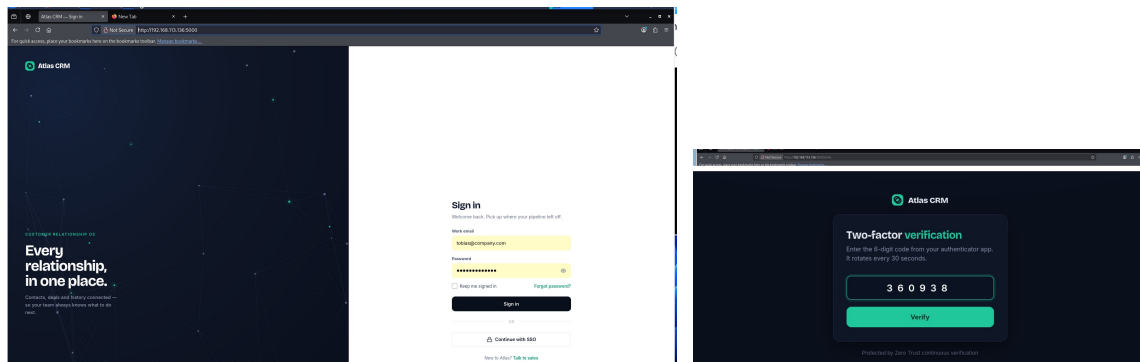
# --- Monitoring VM (.134): start BEFORE Stage 4 ---
python3 alert_listener.py          # listens on 0.0.0.0:5140
```

With the VMs running, Stages 1-2 are driven from a browser on the User VM and Stages 3-4 from a `www-data` shell on the web server. *Stage 1* (Figure 6): sign in at `http://192.168.113.136:5000` with the demo account, then submit the code from `otp_now.py` at `/mfa`. *Stage 2* (Figure 7): on the dashboard (score 12/100), click “Simulate context change”; the score recomputes to 82/100 and redirects to `/reauth`, where a fresh TOTP restores the session. *Stage 3* (Figure 8): as `www-data`, `ssh dbadmin@192.168.113.133` times out by design, whereas `mysql -u webuser -p -h 192.168.113.133` succeeds for `SELECT` but is refused `CREATE USER/DROP`. *Stage 4*: run `policy_engine.py cat www-data /opt/crm/secure/secret.txt` for the PDP verdict, then `secure_cat.py` to see the read blocked and the honeyfile served, with the alert appearing on the monitor within the same second. Table 6 lists the intentionally weak, lab-only credentials; they must never be reused outside this isolated network.

Table 6. Demo-only credentials used by the reproduction (all lab-internal).

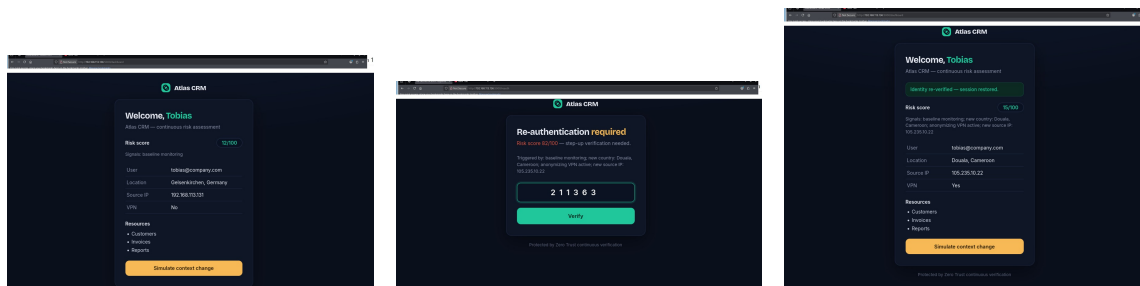
Account	Host	Credential	Defined in
tobias@company.com	Web Server (CRM)	ZeroTrust2026	app.py
TOTP shared secret	Web Server / auth. app	JBSWY3DPEHPK3PXP	app.py / otp_now.py
webuser@...136	Database (MySQL)	Database2026	db_setup.sql
dbadmin	Database (Linux/SSH)	(operator-chosen)	OS install

Figures 6–8 reproduce the screenshots captured during the live run for Stages 1-3 (Stage 4 is shown in Figure 5a in the main text).



(a) Sign-in page posted to Flask's /login route (Werkzeug (b) Server-side TOTP verification at /mfa via pyotp.TOTP.verify()).

Fig. 6. Stage 1 — verified entry: password login and the follow-up TOTP challenge.



(a) Baseline: score 12/100, home IP, no VPN. (b) After /simulate: score $82 \geq 70$; redirected to /reauth. (c) After step-up TOTP: score lowered to 15/100.

Fig. 7. Stage 2 — risk-based re-authentication: baseline context, computed step-up on context change, and the restored session after re-verification.

(a) UFW: 3306/tcp allowed only from the web server, 22/tcp denied from it, SSH open to other admins.

```
database@database-virtual-machine:~$ sudo ufw status
Status: active

To Action From
--
3306/tcp ALLOW 192.168.113.136
22/tcp DENY 192.168.113.136
22 ALLOW Anywhere
22 (v6) ALLOW Anywhere (v6)

database@database-virtual-machine:~$ id dbadmin
uid=1001(dbadmin) gid=1001(dbadmin) groups=1001(dbadmin)
database@database-virtual-machine:~$
```

(b) The one permitted path: webuser reads the customers table and nothing more.

```
database@database-virtual-machine:~$ mysql -u webuser -p -h 192.168.113.133
Enter password:
Warning: Using a password on the command line interface can be insecure.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use crm;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM customers;
+----+-----+-----+
| id | name | email |
+----+-----+-----+
| 1 | Alice | alice@company.com |
| 2 | Bob | bob@company.com |
| 3 | Tobias | tobias@company.com |
| 4 | Alice | alice@company.com |
| 5 | Bob | bob@company.com |
| 6 | Charlie | charlie@company.com |
+----+-----+-----+
6 rows in set (0.01 sec)

mysql>
```

Fig. 8. Stage 3 — micro-segmentation: the SSH pivot is blocked while the single least-privilege data path remains open.

B Complete Laboratory Source Code

This appendix reproduces, in full and unmodified, the source of the laboratory used in Section 6. Each listing below is the actual file used to produce the corresponding figure; none is abridged. Two exceptions are flagged explicitly where they occur: `reauth.html` (B.2.1) and `secure_cat.py` (B.8) were not separately exported as standalone files during the live run and are reconstructed here, in clean and valid form, directly from `app.py`'s own `render_template/evaluate` calls and from the observed output in Figures 7b and 5a every other listing is copied byte-for-byte from the working project directory. Together with Appendix A, this appendix is sufficient to rebuild the entire laboratory.

B.1 `app.py` - Flask application (identity, MFA, risk engine, PDP/PEP routes)

`app.py`

```
#!/usr/bin/env python3
"""
Atlas CRM -- real Zero Trust authentication backend (template-based).
Place in ~/crm/app.py. Templates live in ~/crm/templates/:
    login.html base.html mfa.html dashboard.html reauth.html
pip install flask pyotp werkzeug
Run (keeps http://192.168.113.136):
    sudo ~/crm/venv/bin/python app.py
    (or: python app.py 5000 -> http://192.168.113.136:5000)
"""

import sys, hashlib
from flask import Flask, request, session, redirect, render_template
from werkzeug.security import generate_password_hash, check_password_hash
import pyotp
```

```
app = Flask(__name__)
app.secret_key = "zt-demo-secret-change-me"

TOTP_SECRET = "JBSWY3DPEHPK3PXP" # enrolled in FreeOTP / Google Authenticator
USERS = {
    "tobias@company.com": {
        "pwd_hash": generate_password_hash("ZeroTrust2026"),
        "totp": TOTP_SECRET,
        "baseline": {"ip": "192.168.113.131", "country": "Germany", "city": "Gelsenkirchen",
                     "device": "COMPANY-LAPTOP-01", "vpn": False},
    }
}

STEPUP_THRESHOLD = 70

def client_ctx():
    ua = request.headers.get("User-Agent", "")
    ctx = {"ip": request.remote_addr, "country": "Germany", "city": "Gelsenkirchen",
          "device": "COMPANY-LAPTOP-01", "vpn": False,
          "ua_hash": hashlib.sha256(ua.encode()).hexdigest()[:12]}
    ctx.update(session.get("sim_ctx", {})) # input may be simulated; scoring below is real
    return ctx

def compute_risk(ctx, base):
    score, reasons = 12, ["baseline monitoring"]
    if ctx["country"] != base["country"]:
        score += 40; reasons.append(f"new country: {ctx['city']}, {ctx['country']}")
    if ctx["vpn"]:
        score += 25; reasons.append("anonymizing VPN active")
    if ctx["ip"] != base["ip"]:
        score += 5; reasons.append(f"new source IP: {ctx['ip']}")
    if ctx["device"] != base["device"]:
        score += 20; reasons.append(f"unrecognized device: {ctx['device']}")
    return min(score, 100), reasons

@app.route("/")
def index():
    session.clear()
    err = "Invalid email or password." if request.args.get("error") else None
    return render_template("login.html", error=err)

@app.route("/login", methods=["POST"])
def login():
```

```

email = request.form.get("email", "").strip().lower()
pw = request.form.get("password", "")
u = USERS.get(email)
if not u or not check_password_hash(u["pwd_hash"], pw):
    return redirect("/?error=1")          # REAL password check
session["pending"] = email
return redirect("/mfa")

@app.route("/mfa")
def mfa():
    if "pending" not in session:
        return redirect("/")
    return render_template("mfa.html", error=bool(request.args.get("error")))

@app.route("/mfa", methods=["POST"])
def mfa_check():
    email = session.get("pending")
    if not email:
        return redirect("/")
    if not pyotp.TOTP(USERS[email]["totp"]).verify(request.form.get("otp", ""), valid_window=1):
        return redirect("/mfa?error=1")    # REAL TOTP verification
    session.pop("pending")
    session["user"] = email
    session["ip"] = request.remote_addr
    session["ua"] = client_ctx()["ua_hash"]
    return redirect("/dashboard")

@app.route("/dashboard")
def dashboard():
    email = session.get("user")
    if not email:
        return redirect("/")
    # continuous verification: detect IP / device change since login
    if request.remote_addr != session.get("ip") or client_ctx()["ua_hash"] != session.get("ua"):
        session.setdefault("sim_ctx", {"ip": request.remote_addr})

    ctx = client_ctx()
    score, reasons = compute_risk(ctx, USERS[email]["baseline"])
    if score >= STEPUP_THRESHOLD and not session.get("stepped_up"):
        return redirect("/reauth")

    stepped_up = session.get("stepped_up", False)
    shown = 15 if stepped_up else score

```

```

color = "mint" if shown < 40 else ("amber" if shown < 70 else "red")
return render_template("dashboard.html",
                       email=email, name="Tobias", score=shown, color=color,
                       reasons=reasons, stepped_up=stepped_up, ctx=ctx)

@app.route("/simulate")
def simulate():
    session["sim_ctx"] = {"country": "Cameroon", "city": "Douala", "vpn": True, "ip": "105.235.10.22"}
    session["stepped_up"] = False
    return redirect("/dashboard")

@app.route("/reauth")
def reauth():
    if "user" not in session:
        return redirect("/")
    score, reasons = compute_risk(client_ctx(), USERS[session["user"]]["baseline"])
    return render_template("reauth.html", score=score, reasons=reasons,
                          error=bool(request.args.get("error")))

@app.route("/reauth", methods=["POST"])
def reauth_check():
    email = session.get("user")
    if not email:
        return redirect("/")
    if not pyotp.TOTP(USERS[email]["totp"]).verify(request.form.get("otp", ""), valid_window=1):
        return redirect("/reauth?error=1")
    session["stepped_up"] = True
    return redirect("/dashboard")

if __name__ == "__main__":
    port = int(sys.argv[1]) if len(sys.argv) > 1 else 80
    app.run(host="0.0.0.0", port=port)

```

B.2 Jinja2 templates

templates/base.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">

```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{% block title %}Atlas CRM{% endblock %} · Atlas CRM</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Bricolage+Grotesque:opsz,wght@12
    ↪ ..96,500;12..96,700;12..96,800&family=Inter:wght@400;500;600&display=swap" rel="stylesheet">
<style>
:root{
  --ink:#0A0F1C;
  --panel:#0F1626;
  --border:#1D2740;
  --field:#0B1322;
  --field-border:#26314D;
  --text:#E9ECF5;
  --muted:#8990A8;
  --accent:#1FC79B;
  --accent-ink:#05231B;
  --amber:#F7B955;
  --amber-ink:#2A1D04;
  --red:#E0533D;
}
*{box-sizing:border-box;margin:0;padding:0}
html,body{height:100%}
body{
  font-family:'Inter',system-ui,Arial,sans-serif;
  background:radial-gradient(120% 90% at 50% -10%, #14213B 0%, var(--ink) 55%) fixed;
  color:var(--text);
  min-height:100vh;
  display:grid;
  place-items:center;
  padding:32px 18px;
  -webkit-font-smoothing:antialiased;
}
.shell{width:100%;max-width:400px}

.brand-row{display:flex;align-items:center;gap:11px;justify-content:center;margin-bottom:20px;
  font-family:'Bricolage Grotesque';font-weight:800;font-size:19px;letter-spacing:-.02em}
.glyph{width:26px;height:26px;border-radius:8px;background:var(--accent);position:relative;flex:none}
.glyph::before,.glyph::after{content:"";position:absolute;width:9px;height:9px;border-radius:50%;border:3
    ↪ px solid var(--ink);background:transparent}
.glyph::before{top:4px;left:4px}
.glyph::after{bottom:4px;right:4px}

.card{background:var(--panel);border:1px solid var(--border);border-radius:16px;
  padding:32px 30px;box-shadow:0 18px 50px rgba(0,0,0,.45)}

```



```

h2{font-family: 'Bricolage Grotesque';font-weight:700;font-size:25px;letter-spacing:-.02em;margin-bottom:6
  ↪ px}
.sub{color:var(--muted);font-size:14px;line-height:1.5;margin-bottom:22px}

.mint{color:var(--accent)} .amber{color:var(--amber)} .red{color:var(--red)}

input{width:100%;padding:13px 14px;margin:8px 0;border-radius:11px;
  border:1.5px solid var(--field-border);background:var(--field);color:#fff;
  font:inherit;font-size:16px;transition:border-color .15s, box-shadow .15s}
input:focus{outline:none;border-color:var(--accent);box-shadow:0 0 4px rgba(31,199,155,.14)}
input.otp{text-align:center;letter-spacing:.5em;font-weight:600;font-size:22px;padding-left:.5em}

button{width:100%;padding:14px;border:0;border-radius:11px;background:var(--accent);
  color:var(--accent-ink);font:inherit;font-weight:700;font-size:15px;cursor:pointer;margin-top:8px;
  transition:transform .08s, filter .15s}
button:hover{filter:brightness(1.06)}
button:active{transform:translateY(1px)}

.btn-amber{display:block;text-align:center;text-decoration:none;margin-top:18px;
  padding:13px 16px;border-radius:11px;background:var(--amber);color:var(--amber-ink);
  font-weight:700;font-size:14.5px}
.btn-amber:hover{filter:brightness(1.04)}

.banner{padding:11px 13px;border-radius:10px;margin-bottom:18px;font-size:13.5px;font-weight:500;
  background:#2A1413;color:var(--red);border:1px solid #4a201c}
.banner.ok{background:#0C2A22;color:var(--accent);border-color:#15453a}

.signals{color:var(--muted);font-size:12.5px;line-height:1.5;margin-top:10px}

.risk{display:flex;align-items:center;justify-content:space-between;
  margin:18px 0 4px;font-size:14px;font-weight:600}
.pill{display:inline-block;padding:4px 14px;border-radius:20px;font-weight:800;font-size:14px;
  background:var(--field);border:1px solid var(--field-border)}
.pill.mint{color:var(--accent)} .pill.amber{color:var(--amber)} .pill.red{color:var(--red)}

table{width:100%;border-collapse:collapse;margin-top:18px}
td{padding:9px 4px;border-bottom:1px solid var(--border);font-size:14px}
td:first-child{color:var(--muted);width:38%}

.label{margin-top:20px;font-weight:600;font-size:14px}
ul{margin:8px 0 0;padding-left:18px;color:#CFD5E6;font-size:14px;line-height:1.7}

.footer{text-align:center;color:#5A6178;font-size:12px;margin-top:18px}
</style>
</head>
<body>

```

```

<div class="shell">
  <div class="brand-row"><span class="glyph" aria-hidden="true"></span> Atlas CRM</div>
  <div class="card">
    {% block body %}{% endblock %}
  </div>
  <p class="foot">{% block foot %}Protected by Zero Trust continuous verification{% endblock %}</p>
</div>
</body>
</html>

```

templates/login.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Atlas CRM -- Sign in</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Bricolage+Grotesque:opsz,wght@12
↳ ..96,500;12..96,700;12..96,800&family=Inter:wght@400;500;600&display=swap" rel="stylesheet">
<style>
:root{
  --ink:#0A0F1C;
  --panel:#0F1626;
  --surface:#FFFFFFF;
  --line:#E6E8EE;
  --text:#10172A;
  --muted:#69708A;
  --accent:#1FC79B;
  --accent-deep:#11997A;
  --amber:#F7B955;
  --field:#F6F7F9;
  --danger:#E0533D;
  --radius:14px;
}

*{box-sizing:border-box;margin:0;padding:0}
html,body{height:100%}
body{
  font-family:'Inter',system-ui,sans-serif;
  color:var(--text);
  background:var(--ink);
  display:grid;
  grid-template-columns:1.05fr 1fr;

```

```

    min-height:100vh;
    -webkit-font-smoothing:antialiased;
}

/* ----- BRAND PANEL ----- */
.brand{
    position:relative;
    background:radial-gradient(120% 90% at 15% 10%, #14213B 0%, var(--panel) 45%, var(--ink) 100%);
    color:#fff;
    overflow:hidden;
    display:flex;
    flex-direction:column;
    justify-content:space-between;
    padding:48px 56px;
}
#graph{position:absolute;inset:0;width:100%;height:100%;display:block}
.brand > *{position:relative;z-index:2}

.wordmark{display:flex;align-items:center;gap:11px;font-family:'Bricolage Grotesque';font-weight:800;font
    ↪ -size:21px;letter-spacing:-.02em}
.glyph{width:30px;height:30px;border-radius:9px;background:var(--accent);position:relative;flex:none}
.glyph::before,.glyph::after{content:"";position:absolute;background:var(--ink);border-radius:2px}
.glyph::before{width:11px;height:11px;border-radius:50%;border:3px solid var(--ink);background:
    ↪ transparent;top:5px;left:5px}
.glyph::after{width:11px;height:11px;border-radius:50%;border:3px solid var(--ink);background:transparent
    ↪ ;bottom:5px;right:5px}

.pitch{max-width:30ch}
.eyebrow{font-size:12px;font-weight:600;letter-spacing:.16em;text-transform:uppercase;color:var(--accent)
    ↪ }
.pitch h1{
    font-family:'Bricolage Grotesque';
    font-weight:700;
    font-size:clamp(34px,3.4vw,48px);
    line-height:1.02;
    letter-spacing:-.03em;
    margin:18px 0 16px;
}
.pitch p{color:#A9B0C7;font-size:15px;line-height:1.6;max-width:34ch}

.ticker{display:flex;gap:36px;border-top:1px solid rgba(255,255,255,.1);padding-top:24px}
.ticker .stat .n{font-family:'Bricolage Grotesque';font-weight:700;font-size:24px;letter-spacing:-.02em}
.ticker .stat .l{font-size:12px;color:#8990A8;margin-top:3px}
.ticker .stat .n .u{color:var(--accent)}

/* ----- FORM PANEL ----- */

```

```

.auth{
  background:var(--surface);
  display:flex;
  align-items:center;
  justify-content:center;
  padding:40px 24px;
}
.card{width:100%;max-width:380px}
.card h2{font-family:'Bricolage Grotesque';font-weight:700;font-size:30px;letter-spacing:-.025em}
.card .sub{color:var(--muted);font-size:14.5px;margin-top:8px;line-height:1.5}

/* server-side error banner (from Flask) */
/* removed -- login no longer shows an inline error banner */

form{margin-top:26px}
.field{margin-bottom:18px}
.field label{display:block;font-size:13px;font-weight:600;margin-bottom:7px;color:#37405C}
.input{position:relative}
.input input{
  width:100%;
  font:inherit;font-size:15px;
  padding:13px 14px;
  background:var(--field);
  border:1.5px solid transparent;
  border-radius:11px;
  color:var(--text);
  transition:border-color .15s, background .15s, box-shadow .15s;
}
.input input::placeholder{color:#A6ABBD}
.input input:focus{
  outline:none;
  background:#fff;
  border-color:var(--accent);
  box-shadow:0 0 0 4px rgba(31,199,155,.14);
}
.input.has-toggle input{padding-right:46px}
.toggle{
  position:absolute;top:50%;right:8px;transform:translateY(-50%);
  background:none;border:none;cursor:pointer;
  width:34px;height:34px;border-radius:8px;
  display:grid;place-items:center;color:var(--muted);
}
.toggle:hover{background:#EDEF3;color:var(--text)}
.toggle:focus-visible{outline:2px solid var(--accent);outline-offset:1px}

.field .err{display:none;color:var(--danger);font-size:12.5px;margin-top:6px;font-weight:500}

```

```

.field.invalid input{border-color:var(--danger);background:#fff;box-shadow:0 0 4px rgba(224,83,61,.12)}
.field.invalid .err{display:block}

.row{display:flex;align-items:center;justify-content:space-between;margin:4px 0 24px}
.remember{display:flex;align-items:center;gap:8px;font-size:13.5px;color:#37405C;cursor:pointer;user-
  ↳ select:none}
.remember input{width:17px;height:17px;accent-color:var(--accent-deep);cursor:pointer}
.link{color:var(--accent-deep);font-size:13.5px;font-weight:600;text-decoration:none}
.link:hover{text-decoration:underline}

.btn{
  width:100%;font:inherit;font-weight:600;font-size:15px;
  padding:14px;border:none;border-radius:11px;cursor:pointer;
  background:var(--ink);color:#fff;
  display:flex;align-items:center;justify-content:center;gap:9px;
  transition:transform .08s, background .15s;
}
.btn:hover{background:#1B2741}
.btn:active{transform:translateY(1px)}
.btn:focus-visible{outline:2px solid var(--accent);outline-offset:2px}
.btn[aria-busy="true"]{background:#1B2741;cursor:progress}
.spinner{width:17px;height:17px;border:2.5px solid rgba(255,255,255,.35);border-top-color:#fff;border-
  ↳ radius:50%;animation:spin .7s linear infinite;display:none}
.btn[aria-busy="true"] .spinner{display:block}
.btn[aria-busy="true"] .label{opacity:.85}
@keyframes spin{to{transform:rotate(360deg)}}

.divider{display:flex;align-items:center;gap:14px;margin:26px 0;color:#A6ABBD;font-size:12px;letter-
  ↳ spacing:.05em}
.divider::before,.divider::after{content:"";flex:1;height:1px;background:var(--line)}

.sso{
  width:100%;font:inherit;font-weight:600;font-size:14.5px;
  padding:13px;border:1.5px solid var(--line);border-radius:11px;cursor:pointer;
  background:#fff;color:var(--text);
  display:flex;align-items:center;justify-content:center;gap:9px;
  transition:border-color .15s, background .15s;
}
.sso:hover{background:#FAFAFB;border-color:#D4D8E2}

.signup{text-align:center;margin-top:28px;font-size:13.5px;color:var(--muted)}
.signup a{color:var(--accent-deep);font-weight:600;text-decoration:none}
.signup a:hover{text-decoration:underline}

/* ----- RESPONSIVE ----- */
@media (max-width:880px){

```

```

body{grid-template-columns:1fr}
.brand{padding:30px 28px;min-height:auto}
.brand .pitch h1{font-size:28px;margin:14px 0 12px}
.brand .pitch p{display:none}
.ticker{display:none}
#graph{opacity:.55}
.auth{padding:36px 24px 56px}
}
@media (prefers-reduced-motion:reduce){
  .spinner{animation:none}
}
</style>
</head>
<body>

<section class="brand">
  <canvas id="graph"></canvas>

  <div class="wordmark">
    <span class="glyph" aria-hidden="true"></span>
    Atlas CRM
  </div>

  <div class="pitch">
    <span class="eyebrow">Customer Relationship OS</span>
    <h1>Every relationship, in one place.</h1>
    <p>Contacts, deals and history connected -- so your team always knows what to do next.</p>
  </div>

  <div class="ticker">
    <div class="stat"><div class="n">12.4<span class="u">k</span></div><div class="l">contacts tracked</div>
    ⇨ </div>
    <div class="stat"><div class="n">98<span class="u">%</span></div><div class="l">deals up to date</div>
    ⇨ </div>
    <div class="stat"><div class="n">3.2<span class="u">s</span></div><div class="l">avg. response time</div>
    ⇨ </div>
  </div>
</section>

<main class="auth">
  <div class="card">
    <h2>Sign in</h2>
    <p class="sub">Welcome back. Pick up where your pipeline left off.</p>

    <!-- REAL submission: POST to the Flask /login route -->
    <form id="loginForm" method="POST" action="/login" novalidate>

```

```

<div class="field" id="f-email">
  <label for="email">Work email</label>
  <div class="input">
    <input type="email" id="email" name="email" placeholder="you@company.com" autocomplete="
↪ username" required>
  </div>
  <span class="err" id="e-email">Enter a valid email address.</span>
</div>

<div class="field" id="f-pass">
  <label for="password">Password</label>
  <div class="input has-toggle">
    <input type="password" id="password" name="password" placeholder="....." autocomplete="
↪ current-password" required>
    <button type="button" class="toggle" id="toggle" aria-label="Show password" aria-pressed="false
↪ ">
      <svg id="eye" width="19" height="19" viewBox="0 0 24 24" fill="none" stroke="currentColor"
↪ stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M2 12s3.5-7 10-7 10 7 10
↪ 7-3.5 7-10 7-10-7-10-7Z"/><circle cx="12" cy="12" r="3"/></svg>
    </button>
  </div>
  <span class="err" id="e-pass">Password is required.</span>
</div>

<div class="row">
  <label class="remember"><input type="checkbox" id="remember" name="remember"> Keep me signed in</
↪ label>
  <a href="#" class="link">Forgot password?</a>
</div>

<button type="submit" class="btn" id="submit">
  <span class="spinner" aria-hidden="true"></span>
  <span class="label">Sign in</span>
</button>

<div class="divider">OR</div>

<button type="button" class="sso" id="sso">
  <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2
↪ " stroke-linecap="round" stroke-linejoin="round"><rect x="3" y="11" width="18" height="11" rx="2"/>
↪ <path d="M7 11V7a5 5 0 0 1 10 0v4"/></svg>
  Continue with SSO
</button>

<p class="signup">New to Atlas? <a href="#">Talk to sales</a></p>
</form>

```

```

</div>
</main>

<script>
/* ----- Client-side validation, then REAL POST ----- */
const form = document.getElementById('loginForm');
const email = document.getElementById('email');
const pass = document.getElementById('password');
const submit= document.getElementById('submit');

const toggle = document.getElementById('toggle');
toggle.addEventListener('click', () => {
  const show = pass.type === 'password';
  pass.type = show ? 'text' : 'password';
  toggle.setAttribute('aria-pressed', show);
  toggle.setAttribute('aria-label', show ? 'Hide password' : 'Show password');
});

function setInvalid(id, on){ document.getElementById(id).classList.toggle('invalid', on); }
[email, pass].forEach(el => el.addEventListener('input', () => {
  setInvalid(el === email ? 'f-email' : 'f-pass', false);
}));

form.addEventListener('submit', (e) => {
  const emailOk = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email.value.trim());
  const passOk = pass.value.length > 0;
  setInvalid('f-email', !emailOk);
  setInvalid('f-pass', !passOk);

  // Only block the POST when the form is incomplete.
  if(!emailOk){ e.preventDefault(); email.focus(); return; }
  if(!passOk){ e.preventDefault(); pass.focus(); return; }

  // Valid → DO NOT preventDefault. The browser posts to /login,
  // Flask checks the password and redirects to /mfa.
  submit.setAttribute('aria-busy','true');
  submit.querySelector('.label').textContent = 'Signing in...';
});

document.getElementById('sso').addEventListener('click', () => {
  alert('SSO is not wired up in this demo backend.');
```



```

const ctx = canvas.getContext('2d');
const reduce = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
let w, h, dpr, nodes = [], edges = [], pulses = [];

function resize(){
  dpr = Math.min(window.devicePixelRatio || 1, 2);
  const r = canvas.getBoundingClientRect();
  w = r.width; h = r.height;
  canvas.width = w * dpr; canvas.height = h * dpr;
  ctx.setTransform(dpr,0,0,dpr,0,0);
  build();
}

function build(){
  const count = Math.max(10, Math.round(w*h/42000));
  nodes = Array.from({length:count}, () => ({
    x: Math.random()*w, y: Math.random()*h,
    vx:(Math.random()-0.5)*.18, vy:(Math.random()-0.5)*.18,
    r: Math.random()*2.4 + 1.6, hot: Math.random() < .22
  }));
  edges = [];
  for(let i=0;i<nodes.length;i++){
    for(let j=i+1;j<nodes.length;j++){
      const d = Math.hypot(nodes[i].x-nodes[j].x, nodes[i].y-nodes[j].y);
      if(d < Math.min(w,h)*0.34) edges.push([i,j]);
    }
  }
  pulses = [];
}

function spawnPulse(){
  if(edges.length && pulses.length < 5)
    pulses.push({ e: edges[(Math.random()*edges.length)|0], t: 0, sp: .006 + Math.random()*0.006 });
}

function draw(){
  ctx.clearRect(0,0,w,h);
  if(!reduce){
    nodes.forEach(n => {
      n.x += n.vx; n.y += n.vy;
      if(n.x < 0 || n.x > w) n.vx *= -1;
      if(n.y < 0 || n.y > h) n.vy *= -1;
    });
  }
  edges.forEach(([a,b]) => {
    const A = nodes[a], B = nodes[b];
    ctx.strokeStyle = 'rgba(140,160,210,0.10)'; ctx.lineWidth = 1;
    ctx.beginPath(); ctx.moveTo(A.x,A.y); ctx.lineTo(B.x,B.y); ctx.stroke();
  });
  for(let i=pulses.length-1;i>=0;i--){

```

```

    const p = pulses[i];
    const A = nodes[p.e[0]], B = nodes[p.e[1]];
    p.t += p.sp;
    if(p.t >= 1){ pulses.splice(i,1); continue; }
    const x = A.x + (B.x-A.x)*p.t, y = A.y + (B.y-A.y)*p.t;
    ctx.beginPath(); ctx.fillStyle = 'rgba(31,199,155,0.95)';
    ctx.arc(x,y,2.6,0,Math.PI*2); ctx.fill();
  }
  nodes.forEach(n => {
    ctx.beginPath();
    ctx.fillStyle = n.hot ? 'rgba(31,199,155,0.9)' : 'rgba(170,185,225,0.45)';
    ctx.arc(n.x,n.y,n.r,0,Math.PI*2); ctx.fill();
    if(n.hot){
      ctx.beginPath(); ctx.strokeStyle = 'rgba(31,199,155,0.25)'; ctx.lineWidth = 1;
      ctx.arc(n.x,n.y,n.r+5,0,Math.PI*2); ctx.stroke();
    }
  });
  if(!reduce){ if(Math.random() < .04) spawnPulse(); requestAnimationFrame(draw); }
}
window.addEventListener('resize', resize);
resize(); draw();
})();
</script>
</body>
</html>

```

templates/mfa.html

```

{% extends "base.html" %}
{% block title %}Two-factor verification{% endblock %}
{% block body %}
  <h2>Two-factor <span class="mint">verification</span></h2>
  <p class="sub">Enter the 6-digit code from your authenticator app. It rotates every 30 seconds.</p>

  {% if error %}
  <div class="banner">Wrong or expired code -- enter the current one.</div>
  {% endif %}

  <form method="post" action="/mfa">
    <input class="otp" name="otp" inputmode="numeric" autocomplete="one-time-code"
      maxlength="6" pattern="[0-9]*" placeholder="000000" autofocus>
    <button type="submit">Verify</button>
  </form>
{% endblock %}

```

templates/dashboard.html

```
{% extends "base.html" %}
{% block title %}Dashboard{% endblock %}
{% block body %}
  <h2>Welcome, <span class="mint">{{ name }}</span></h2>
  <p class="sub">Atlas CRM -- continuous risk assessment</p>

  {% if stepped_up %}
  <div class="banner ok">Identity re-verified -- session restored.</div>
  {% endif %}

  <div class="risk">
    <span>Risk score</span>
    <span class="pill {{ color }}">{{ score }}/100</span>
  </div>
  <p class="signals">Signals: {{ reasons | join("; ") }}</p>

  <table>
    <tr><td>User</td><td>{{ email }}</td></tr>
    <tr><td>Location</td><td>{{ ctx.city }}, {{ ctx.country }}</td></tr>
    <tr><td>Source IP</td><td>{{ ctx.ip }}</td></tr>
    <tr><td>VPN</td><td>{{ "Yes" if ctx.vpn else "No" }}</td></tr>
  </table>

  <p class="label">Resources</p>
  <ul>
    <li>Customers</li>
    <li>Invoices</li>
    <li>Reports</li>
  </ul>

  <a class="btn-amber" href="/simulate">Simulate context change</a>
{% endblock %}
```

B.2.1 templates/reauth.html - step-up re-authentication (reconstructed). Not separately exported during the live run; reconstructed here from `app.py`'s `render_template("reauth.html", score=score, reasons=reasons, error=...)` call and from the structure of `mfa.html`, which it mirrors exactly. Its rendered output is shown in Figure 7b.

templates/reauth.html

```
{% extends "base.html" %}
{% block title %}Re-authentication required{% endblock %}
{% block body %}
  <h2>Re-authentication <span class="amber">required</span></h2>
```

```

<p class="sub"><span class="red">Risk score {{ score }}/100</span> -- step-up verification needed.</p>
<p class="signals">Triggered by: {{ reasons | join("; ") }}</p>

{% if error %}
<div class="banner">Wrong or expired code -- enter the current one.</div>
{% endif %}

<form method="post" action="/reauth" style="margin-top:16px">
  <input class="otp" name="otp" inputmode="numeric" autocomplete="one-time-code"
    maxlength="6" pattern="[0-9]*" placeholder="000000" autofocus>
  <button type="submit">Verify</button>
</form>
{% endblock %}

```

B.3 policy_engine.py - Policy Decision Point (PDP)

policy_engine.py

```

#!/usr/bin/env python3

import sys
import os
import datetime
import socket

CONTROL_IP = "192.168.113.134"
CONTROL_PORT = 5140

SENSITIVE_FILES = {
    "/opt/crm/secure/secret.txt"
}

SENSITIVE_NAMES = {
    os.path.basename(p)
    for p in SENSITIVE_FILES
} | {"secret.txt"}

SERVICE_ACCOUNTS = {
    "www-data",
    "apache",
    "nobody"
}

ALLOWED_READERS = {
    "root",

```

```
    "crmdadmin"
}

COLLECTION_TOOLS = {
    "cat",
    "less",
    "more",
    "head",
    "tail",
    "cp",
    "scp",
    "curl",
    "wget",
    "base64",
    "dd",
    "xxd"
}

THRESHOLD = 70

def send_alert(verdict):

    try:

        sock = socket.socket(
            socket.AF_INET,
            socket.SOCK_STREAM
        )

        sock.connect(
            (CONTROL_IP, CONTROL_PORT)
        )

        msg = (
            f"HOST=WEB-SERVER | "
            f"USER={verdict['user']} | "
            f"PROCESS={verdict['process']} | "
            f"TARGET={verdict['target']} | "
            f"RISK={verdict['risk']} | "
            f"DECISION={verdict['decision']}"
        )

        sock.sendall(
            (msg + "\n").encode()
        )
```

```
sock.close()

except Exception as e:

    print(
        f"Unable to send alert: {e}"
    )

def evaluate(user, process, target):

    score = 0

    reasons = []

    sensitive = (
        target in SENSITIVE_FILES
        or
        os.path.basename(target)
        in SENSITIVE_NAMES
    )

    if sensitive:

        score += 40

        reasons.append(
            "target is a classified asset"
        )

    if user in SERVICE_ACCOUNTS:

        score += 30

        reasons.append(
            f"unusual actor: service account '{user}'"
        )

    if sensitive and user not in ALLOWED_READERS:

        score += 10

        reasons.append(
            "actor not in allow list"
        )
```

```
if (
    process in COLLECTION_TOOLS
    and sensitive
):

    score += 10

    reasons.append(
        f"collection tool: {process}"
    )

score = min(score, 100)

return {

    "user": user,

    "process": process,

    "target": target,

    "risk": score,

    "decision":
        "DENY"
        if score >= THRESHOLD
        else "ALLOW",

    "reasons":
        reasons
        if reasons
        else ["no policy matched"]
}

def main():

    if len(sys.argv) == 4:

        process = sys.argv[1]
        user = sys.argv[2]
        target = sys.argv[3]

    else:
```

```
process = "cat"

user = "www-data"

target = "/opt/crm/secure/secret.txt"

v = evaluate(
    user,
    process,
    target
)

if v["decision"] == "DENY":

    send_alert(v)

print()

print(
    "=== POLICY DECISION POINT (PDP) ==="
)

print(
    f"timestamp : "
    f"{datetime.datetime.now().isoformat(timespec='seconds')}"
)

print(
    f"process   : {v['process']}"
)

print(
    f"user      : {v['user']}"
)

print(
    f"target     : {v['target']}"
)

print(
    "reasons   : "
    + " | ".join(v["reasons"])
)

print(
    f"RISK       : {v['risk']}/100"
```



```
)

print(
    f"DECISION : {v['decision']}"
)

sys.exit(
    0
    if v["decision"] == "ALLOW"
    else 3
)

if __name__ == "__main__":
    main()
```

B.4 alert_listener.py - monitoring console

alert_listener.py

```
#!/usr/bin/env python3

import socketserver
import datetime

class Handler(socketserver.StreamRequestHandler):

    def handle(self):

        line = self.rfile.readline().decode(errors="replace").strip()

        if line:

            ts = datetime.datetime.now().strftime("%H:%M:%S")

            print("\n" + "="*70)
            print(f"[{ts}] SECURITY ALERT")
            print("="*70)
            print(line)
            print("="*70 + "\n", flush=True)

if __name__ == "__main__":

    print(
        "SysFlow monitoring -- listening for alerts on 0.0.0.0:5140"
```

```

)

with socketserver.ThreadingTCPServer(
    ("0.0.0.0",5140),
    Handler
) as srv:

    srv.serve_forever()

```

B.5 db_setup.sql - database schema and least-privilege account

db_setup.sql

```

-- Run on the Database VM (192.168.113.133):  sudo mysql < db_setup.sql
CREATE DATABASE IF NOT EXISTS crm;
USE crm;

CREATE TABLE IF NOT EXISTS customers (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    email VARCHAR(100)
);
INSERT INTO customers (name, email) VALUES
    ('Alice', 'alice@acme.io'),
    ('Bob', 'bob@acme.io'),
    ('Charlie', 'charlie@acme.io');

-- Least privilege: the web app account may ONLY do CRUD on crm.*, and ONLY from the Web Server.
CREATE USER IF NOT EXISTS 'webuser'@'192.168.113.136' IDENTIFIED BY 'Database2026';
GRANT SELECT, INSERT, UPDATE, DELETE ON crm.* TO 'webuser'@'192.168.113.136';
-- no CREATE USER / DROP / GRANT, no access to the mysql.* system schema, no other host.
FLUSH PRIVILEGES;

```

B.6 honeyfile.txt - decoy served on DENY

honeyfile.txt

```

=====

TOP SECRET

CEO Salary : 1000 EUR

Database Password :

```

```
fakepassword123
```

```
=====
```

B.7 otp_now.py - live TOTP code for manual reproduction

otp_now.py

```
#!/usr/bin/env python3
"""Prints the current valid TOTP code (read it live like an authenticator app).
Usage: python3 otp_now.py
"""
import pyotp
print("Current OTP:", pyotp.TOTP("JBSWY3DPEHPK3PXP").now(), "(rotates every 30s)")
```

B.8 secure_cat.py - Policy Enforcement Point (PEP) wrapper (reconstructed)

Not separately exported during the live run; reconstructed here from the observed console output in Figure 5a ([PDP] risk=90/100, [PDP] decision=DENY, [PEP] Access blocked, Redirecting attacker to Honeyfile..., followed by the verbatim contents of honeyfile.txt) and from policy_engine.py's own evaluate/send_alert interface, which it calls directly.

secure_cat.py

```
#!/usr/bin/env python3
"""
secure_cat.py -- Policy Enforcement Point (PEP) wrapper around a sensitive read.
Calls the PDP (policy_engine.evaluate) for a verdict, then enforces it:
    ALLOW -> print the real file
    DENY   -> log the attempt (via policy_engine.send_alert) and
              transparently serve the decoy honeyfile.txt instead
Run: python3 secure_cat.py [process] [user] [target]
    (with no arguments, defaults to the Stage-4 scenario:
    cat / www-data / /opt/crm/secure/secret.txt)
"""
import sys
from policy_engine import evaluate, send_alert

SECRET_FILE = "/opt/crm/secure/secret.txt"
HONEYFILE = "honeyfile.txt"

def secure_cat(process="cat", user="www-data", target=SECRET_FILE):
    verdict = evaluate(user, process, target)
```

```
print(f"[PDP] risk={verdict['risk']}/100")
print(f"[PDP] decision={verdict['decision']}")
print()

if verdict["decision"] == "DENY":
    send_alert(verdict)
    print("[PEP] Access blocked")
    print()
    print("Redirecting attacker to Honeyfile...")
    print()
    with open(HONEYFILE) as f:
        print(f.read())
else:
    print("[PEP] Access granted")
    with open(target) as f:
        print(f.read())

if __name__ == "__main__":
    args = sys.argv[1:]
    if len(args) == 3:
        secure_cat(*args)
    else:
        secure_cat()
```